PORTFOLIO PRODUCT STRATEGY

GitQuest

Break it. Fix it. Prove it.



A zero-cost, local-first DevSecOps incident simulator where players investigate compromised repositories, repair delivery controls, and produce verifiable evidence.

CURRENT STATE + NEXT IMPLEMENTATION PHASES

Prepared for James Maherry | 19 October 2026 | v0.4 baseline

The product decision

GitQuest is no longer positioned as a broad command tutorial. Its strongest identity is a defensive engineering game that turns remediation work into portfolio evidence.

A free, local-first DevSecOps simulator where the player contains a software supply-chain incident and proves the repaired state with deterministic checks.

Why this direction wins

- **Cloud engineering:** delivery pipelines, artifacts, IaC, containers, and optional local Kubernetes.
- **Cybersecurity:** investigation, secret exposure, workflow risk, policy, and incident response.
- **DevSecOps:** security controls become automated, testable parts of the delivery system.
- **Portfolio signal:** reviewers can inspect architecture, threat models, tests, CI, and generated evidence.

Scope rule

A proposed feature must improve the playable incident loop, the evidence trail, or the hiring demonstration. If it does none of those, it waits.

Keep	Deepen	Defer
Narrative, missions, Rich terminal UI	Real-state validation and debriefs	Long command-recall sequences
Local execution and disposable labs	Attempt history and evidence integrity	Hosted arbitrary code execution
XP and campaign personality	Progress based on verified outcomes	Leaderboards and social features

What is implemented now

The first three product increments established a safe runtime, an evidence-driven scenario, and a browser interface over shared application services.

Capability	State	Why it matters
Installable CLI	Complete	Conventional packaging and a stable gitquest command
Managed workspaces	Complete	Marked deletion boundary, traversal and symlink defenses
Runtime profiles	Complete	Lite works with Python and Git; later tooling is optional
Scenario manifests	Complete	Versioned data drives briefing, scoring, and evidence
Supply Chain Siege	Complete	A playable compromised-repository vertical slice
Evidence reports	Complete	Markdown for people and JSON for automation
Browser console	Complete	Visual delivery without duplicating scenario logic
CI and Dependabot	Complete	The repository starts proving its own delivery claims

Current player loop

1. Brief	2. Investigate	3. Remediate	4. Prove	5. Debrief
Read incident context	Inspect Git and files	Repair the controls	Run deterministic checks	Review score and evidence

Honest boundary

The repository currently has one evidence-driven security scenario. Scanner adapters, SQLite attempt history, pipeline missions, containers, IaC, and Kubernetes remain planned. They should not be presented as implemented until tests and evidence exist.

Experience and visual system

The browser console now uses a strict monochrome interface: product-like enough for a portfolio demo, restrained enough to keep incident information dominant.

Token	Value	Use
Background	#0A0A0A	Page canvas
Surface	#141414	Workspaces, scores, notices
Primary text	#FFFFFF	Headlines, actions, selected state
Secondary text	#A3A3A3	Metadata and supporting copy
Border	#262626	Structure without decorative shadows

Interface principles

- The GitQuest logo is packaged locally; no font, image, script, or analytics request leaves the machine.
- Large type establishes hierarchy while objective rows remain compact and operational.
- State is expressed with words, borders, fill, and shape rather than color alone.
- Visible focus, semantic headings, a skip link, responsive layouts, and reduced-motion support are baseline requirements.
- Buttons perform real local actions; the interface contains no decorative controls.

Deployment boundary

The browser is a local operations console, not a public web service. A future portfolio showcase should be static and sanitized. It must never expose the workspace API or execute visitor-controlled code.

Architecture that can grow

The domain does not know whether a player uses the terminal or browser, and it does not know whether future evidence comes from Git, Gitleaks, Trivy, Podman, or Kubernetes.

Layer	Responsibility	Boundary
Interfaces	Terminal rendering, HTTP routing, browser assets	Calls application services only
Application	Catalog, start, verify, score, report use cases	Coordinates without shell or UI logic
Domain	Scenario, objective, check, score models	Pure and deterministic Python
Runtime	Workspace creation, safety marker, reset	Owns destructive workspace operations
Adapters	Git and future tool execution	Normalizes external results
Persistence	Player data and planned attempt records	Versioned local schema
Reporting	Human and machine evidence	Never copies secrets or source contents

Dependency direction

Browser + terminal -> application services -> domain rules

Runtime, adapters, persistence, and reporting sit behind the application layer. New tools should enter through typed adapters so scanner-specific output does not leak into scoring or interface code.

Migration approach

Keep the legacy campaign working while scenarios become the dominant content model. Progression and persistence should migrate incrementally, with explicit schema versions and tests for existing save data.

Security model for a security game

Scenario content, player-edited repositories, and browser requests all cross trust boundaries. The safest default is local, disposable, least privilege, synthetic data, and no network.

Threat	Required control	Verification
Workspace escape	Resolved managed root, safety marker, traversal and symlink rejection	Adversarial unit tests
Command injection	Argument arrays, no shell strings from manifests, timeouts	Adapter and fixture tests
Credential exposure	Fake tokens only; no secret values in reports	Report assertions
Cross-site localhost request	Custom mutation header and no CORS	HTTP integration test
DNS rebinding	Strict localhost Host validation	HTTP integration test
Browser injection	textContent and restrictive CSP	Code review and browser checks
Unsafe future containers	Rootless runtime, limits, no host socket	Standard-profile policy tests

Residual risk

- Another process running as the same local user can call the localhost API.
- Regex checks prove declared outcomes, not complete real-world security.
- Removing a secret from the current tree does not purge history or rotate a real credential.
- The development HTTP server must not be exposed to a network or treated as multi-user.

Delivery system: local and hosted parity

The Makefile is the contract between a developer laptop and GitHub Actions. CI calls the same quality gate instead of maintaining a second set of commands.

Make target	Outcome
install / install-dev	Editable app installation with optional quality tools
run / web	Terminal campaign or localhost incident console
doctor / scenarios	Capability and content discovery
scenario-start / verify	Repeatable incident workflow
test	Dependency-free unit and integration suite
lint / format-check	Ruff static and formatting gates
build	Wheel and source distribution
ci	Lint, format, tests, and package builds

CI security choices

- Read-only repository permission and a ten-minute job timeout.
- Checkout credentials are not persisted after source retrieval.
- GitHub-authored actions are pinned to full immutable commit SHAs.
- Python 3.9 preserves the current local baseline; Python 3.13 proves forward compatibility.
- Dependabot reviews Python and GitHub Actions weekly in grouped pull requests.

Cost posture

Hosted checks are convenient but non-essential. Every important gate remains runnable locally, and no workflow provisions cloud resources, deploys infrastructure, or requires stored credentials.

v0.5 - attempts and evidence

The next highest-value increment is persistence around the scenario engine. It turns a single score into a history of repeatable, inspectable incident responses.

Implementation slice

- Add SQLite attempt, objective-result, and artifact metadata tables.
- Record scenario version, timestamps, Git commit ID, score, pass state, and report hashes.
- Create a small migration mechanism and repository interface.
- Add attempt history to the browser and terminal without coupling the domain to SQLite.
- Add safe report view/download routes indexed internally; never accept arbitrary paths.
- Publish one sanitized sample report without machine-specific paths.

Exit criteria

Requirement	Proof
Two attempts can be compared	Integration test plus browser history view
Report deletion does not corrupt history	Persistence test with missing artifact
No arbitrary file read	Traversal and unknown-report HTTP tests
Evidence identity is stable	Hash and commit assertions
Schema can evolve	Migration test from the first database version

This should be the next feature commit after the current UI and automation increment.

v0.6 - one campaign model

GitQuest's original game personality remains valuable. The refactor should connect it to evidence-driven incidents instead of maintaining two unrelated products.

Manifest additions

Field	Purpose
prerequisites	Unlock order and expected knowledge
skills	Portfolio and progression mapping
estimated_minutes	Expectation setting
hints	Tiered help without immediately revealing the repair
debrief	Root cause, real-world response, and tradeoffs
campaign	Track and sequence

Progression rules

- Award completion and XP only from verified objective results.
- Preserve valid alternative workflows by grading final state rather than command history.
- Show locked, ready, active, failed, and completed states with text and shape.
- Version save data and test migration from the current JSON structure.

Player outcome

A new user should know which incident to attempt, what runtime profile it needs, how long it may take, why a control failed, and what skill the successful evidence demonstrates.

v0.7 - Pipeline Fortress

The repository's own automation becomes a playable example. Players repair risky CI workflows and GitQuest checks those controls without executing untrusted workflow content.

Incident	Learning outcome	Evidence
Mutable action	Pin an action to an immutable full SHA	Parsed action reference
Permission sprawl	Set read-only defaults and narrow job elevation	Permission model result
Unsafe trigger	Recognize privileged execution of fork content	Trigger and checkout policy
Script injection	Move untrusted expressions into environment variables	Workflow structure check
Persisted token	Disable credentials when no later push is required	Checkout input result

Design constraint

Prefer a narrowly defined parser and explicit checks over running workflow content. The same policy suite should validate GitQuest's CI so the project demonstrates the controls it teaches.

Exit criteria

- At least three Lite-profile pipeline incidents with known failing and passing fixtures.
- Clear findings that name the risky control and expected secure state.
- No evaluator path invokes Actions, shells, or user-provided commands.

v0.8 and v0.9 - tools and artifacts

Tool integrations should enrich a proven scenario engine. They must be optional, bounded, normalized, and unable to make the deterministic grade depend on a changing online database.

Typed adapter contract

State	Required behavior
Ready / clean	Return normalized evidence and tool version
Ready / findings	Return stable finding categories without leaking secrets
Missing	Explain install path and disable only related missions
Timeout	Terminate work and return bounded diagnostic output
Parse failure	Keep raw output local; report a typed adapter error

Recommended sequence

- **Gitleaks:** secret-history investigation and rotation debrief.
- **Trivy filesystem:** dependency and configuration context.
- **CycloneDX:** SBOM generation tied to the evidence record.
- **Rootless Podman:** reproducible OCI build and runtime hardening.
- **Trivy image + Cosign:** scan, sign, and verify one artifact digest.

Safety

Scrub credential environment variables, cap time and output, avoid host sockets and privileged containers, and keep all network-assisted findings explicitly separate from the stable score.

v1.0 - Operation Nightfall

The capstone is one coherent delivery-chain incident, not a collection of disconnected tool demos. Lite must remain complete; container, IaC, and cluster stages are optional profiles.

Seeded failures

- An exposed synthetic credential and evidence of how it entered the repository.
- A mutable, over-privileged CI workflow with an unsafe release step.
- A deterministic dependency-risk fixture.
- A weak container posture and missing artifact identity.
- A policy-violating IaC or Kubernetes manifest for optional Full-profile work.

Required evidence bundle

Lite	Standard	Full
Git findings, workflow policy, remediation commits, scored report	SBOM, image digest, scan result, signature verification	IaC policy, deployment state, cluster hardening checks

Release definition

- Clean-machine Lite setup under ten minutes and pull-request CI under ten minutes.
- Known failing and passing state for every scenario.
- A wheel, source distribution, checksums, sample evidence, threat models, and ADRs.
- One independent reviewer can complete onboarding without developer assistance.

Portfolio launch plan

The project should be understandable at three depths: a recruiter skims for 30 seconds, a hiring manager watches for two minutes, and an engineer inspects for twenty minutes.

Audience	First proof	Deeper proof
Recruiter	One-line pitch, logo, screenshot, outcome	Short demo and release
Cloud / platform	Local delivery architecture	IaC, containers, policy, teardown, cost model
Security	Threat model and attack-defend scenario	Workflow, secrets, evidence integrity
Senior engineer	Tests, package, CI, modular code	ADRs, failure modes, migrations, contracts

90-second demo

- 0-15 seconds: show the incident briefing and failing 0/100 state.
- 15-45 seconds: inspect the compromised Git/workflow/release state.
- 45-65 seconds: show the remediation diff and commit.
- 65-80 seconds: verify the incident and display objective-level results.
- 80-90 seconds: open the sanitized evidence report and CI checks.

Claims discipline

Use measured setup time, CI duration, test count, scenario count, and reviewer outcomes. Do not claim scanner, container, IaC, Kubernetes, signing, or cloud coverage until the corresponding code, tests, and evidence are present.

The next milestone is not more visual polish. It is persistent attempts and evidence integrity, followed by one unified campaign model.